

App Development Project Report

App Dev Project Report

1. Student Details

Name: Arslan Ansari

Roll Number: 25f2001880

Email: 25f2001880@ds.study.iitm.ac.in

About Me: I am a Diplom level student at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

Adventure organizations require efficient systems to manage trekking activities involving trek organizers, staff, and participants. Currently, many trekking groups rely on spreadsheets, phone calls, or manual coordination, which makes it difficult to manage trek approvals, track bookings, avoid overbooking, and maintain trek history.

Approach:

The **Trekking Management Application** is built using **Flask** as the backend framework with a role-based structure for Admin, Staff, and Trekker users. Admins can create and assign treks to staff, while staff can manage assigned treks, update trek statuses, and view participants. Trekkers can browse available treks, view trek details, make bookings, and track their booking history. **Flask-SQLAlchemy** is used for database management and relationships between users, treks, staff, and bookings, while **Jinja2, HTML, and CSS** are used to create dynamic dashboards and interfaces for each role.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing some css code and also took help on some route definitions, also took help in optimising the code and improving variable naming consistency.

The extent of AI/LLM usage is around **15–20%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework present in app.py
SQLAlchemy	Object Relational Mapper for SQLite database present in models.py
Jinja2	Template engine for rendering dynamic HTML pages used in all 21 templates files
Bootstrap 5	Frontend styling and responsive design has been done in base.html
HTML5/CSS3	Web structure and custom page layouts
Flask-Login	User authentication and session management
Werkzeug	Secure password hashing and verification
SQLite	Lightweight local database for storing user data

5. Database Schema / ER Diagram

Tables:

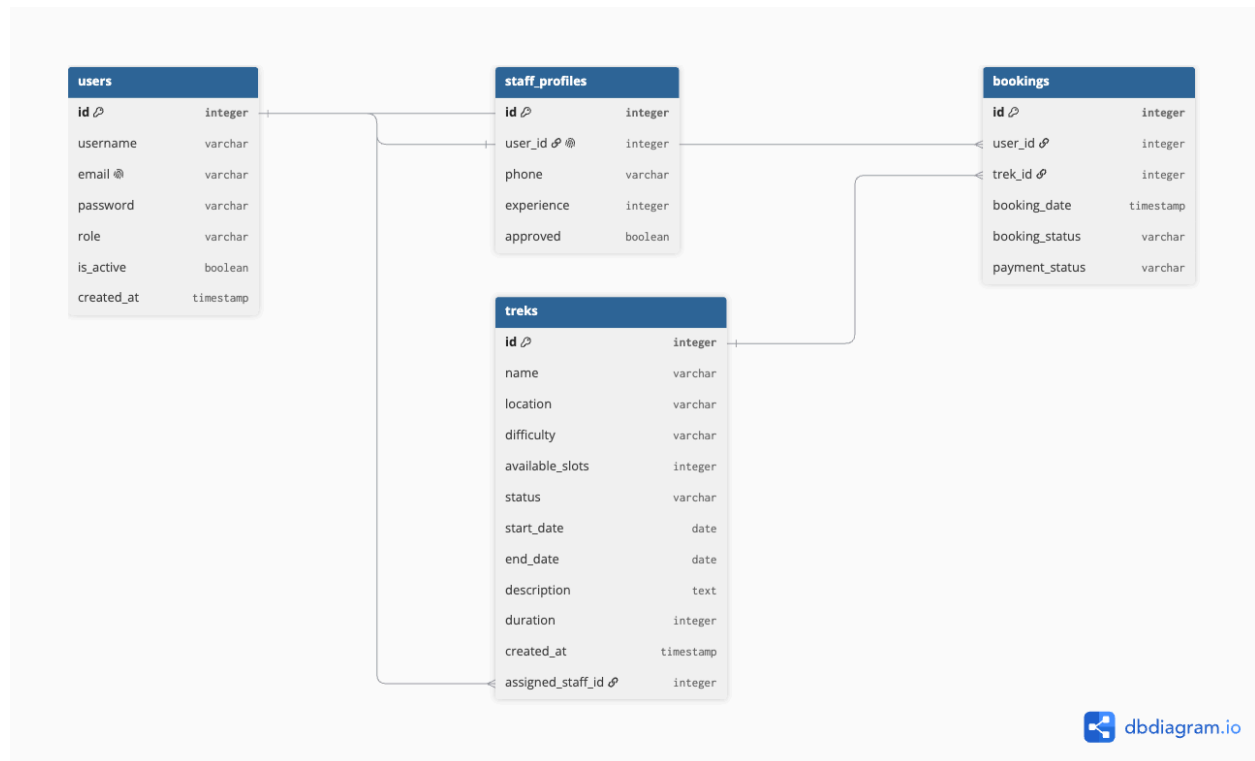
1. **User** — stores user account and profile details (id, username, email, password, role, active status)
2. **StaffProfile** — stores staff-specific details (id, user_id, phone, experience, approval status)
3. **Trek**— stores trekking information (id, name, location, difficulty, available slots, status, start date, end date, description, duration, assigned staff)

4. **Booking** — stores trek booking details (id, user_id, trek_id, booking date, booking status, payment status)

Relationships:

- **One-to-One** → User → StaffProfile
- **One-to-Many** → User → Trek (as assigned staff)
- **One-to-Many** → User → Booking
- **One-to-Many** → Trek → Booking
-

(Insert ER diagram image here – can be created using dbdiagram.io or draw.io or any other website)



(example taken form dbdiagram.io)

6. API Resource Endpoints

Not Applicable

I haven't used api in this project.

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – main Flask application entry point and Flask Blueprints for user and activity routes
- **/models** – database models using SQLAlchemy
- **/templates** – Jinja2 HTML templates
- **/static** – CSS

Implemented Features:

- **Role-based** user registration and login for Admin Staff and Trekkers
 - **Admin dashboard** for managing users, staff approvals, treks, and bookings
 - **Trek creation and staff assignment** by Admin
 - **Staff trek management** with trek status and available slot updates
 - **Participant management** for staff members
 - **Trek browsing and details** for Trekkers
 - **Trek booking system** with booking confirmation and slot management
 - **Overbooking prevention** based on available slots
 - **User booking history** and booking status tracking
 - **Profile management** for users and staff
 - **Role-based route protection** using Flask sessions and custom authentication
 - **Password hashing** for secure password storage
-

8. Video Presentation

Drive Link:

 25F2001880.mp4
